

Laboratorio 4 — AMQP + tercer protocolo (HTTPS) vs MQTT hacia IoT Central

Oct 5, 2026 · @Juan

Estudiante: Juan Camilo Rojas Guerrero · UNAB · IoT + Cloud + Sistemas Distribuidos ·
Repo: `labIotJuanRojas4` · Plantilla `ContenedorVacunas` (temperatura, humedad, nivel_bateria)

1. Rol de AMQP en Azure

AMQP 1.0 es el protocolo nativo del plano de servicio de Azure: Service Bus, Event Hubs y el endpoint interno de IoT Hub (compatible con Event Hubs) hablan AMQP. En el plano de dispositivo IoT Hub/Central lo acepta en el puerto 5671 (TLS) o sobre WebSockets en el 443.

La diferencia con MQTT no es el cifrado ni la autenticación (ambos usan TLS + SAS token), sino el modelo:

- **Conexión → sesiones → links.** Una sola conexión TCP multiplexa varias sesiones y links (telemetría, C2D, twin); en un gateway, varios dispositivos pueden compartir una conexión (multiplexing), cosa que MQTT no permite.
- **Control de flujo por créditos.** El receptor concede N créditos y el emisor no puede enviar más: el hub regula al dispositivo sin cerrar el socket. MQTT no tiene backpressure.
- **Disposición por mensaje.** Cada mensaje vuelve `accepted`, `rejected` (con causa de error) o `released`. En MQTT el error típico es la desconexión (como vimos con QoS 2 en el Lab 3).
- **Coste:** handshake más largo (TLS + SASL + Open + Begin + Attach + Flow) y cabeceras binarias más ricas; tiene sentido en conexiones largas, no en dispositivos que se conectan y se van.

Hallazgo importante: el SDK `azure-iot-device` de Python **no soporta AMQP** (solo MQTT y MQTT-WS; lo dice la documentación oficial de IoT Hub). El parámetro `AMQP/AMQP_WS` existe en los SDK de C#, Java y Node. Por eso el cliente AMQP se implementó directamente con Apache Qpid Proton, igual que en el Lab 3 se hizo MQTT a mano con `paho-mqtt`.

2. Etapa 1 — AMQP hacia IoT Central

`python-vm/device_amqp.py` envía las 3 variables al mismo dispositivo `Vacuna-VM-Python` del Lab 3 cada 5 s y registra latencia y tamaño en `evidencias/amqp_*.jsonl`.

Parámetro	Valor medido (6 oct 2026, GitHub Codespaces)
Cliente	python-qpuid-proton 0.40 (AMQP 1.0)
Host / puerto	<code>iotc-20ba21f7-4249-41e6-bb9c-55e4a8891e49.azure-devices.net:5671</code> (amqps)
Hub obtenido	vía DPS con ID scope <code>0ne00FE097D</code> (<code>provision.py</code>)
Autenticación	SASL PLAIN · usuario <code>Vacuna-VM-Python@sas.iotc-20ba21f7-...</code> · contraseña = SAS token (mismo HMAC-SHA256 del Lab 3)
Link	sender a <code>/devices/Vacuna-VM-Python/messages/events</code>
Propiedades	<code>content-type: application/json</code> , <code>content-encoding: utf-8</code>
Crédito inicial del hub	50 mensajes (<code>link-credit=0x32</code> en la trama <code>@flow</code>)
Handshake completo (TCP+TLS+SASL+Open)	680 ms (rango en 4 corridas: 660–717 ms)
Payload JSON	57–58 B
Mensajes aceptados	46 / 46 (<code>@disposition ... state=@accepted</code>), 0 rechazados
Latencia por mensaje (corrida de 20)	mediana 206,4 ms · p95 213,9 ms · mín 203,1 · máx 223,2
¿Visible en Raw data?	Sí: Temperatura, Humedad y Nivel de Batería cada 5 s; dispositivo en estado Connected (Figura 2)

Qué cambia respecto al MQTT del Lab 3:

- **Conexión:** también persistente, pero se abre en más pasos (Open, Begin, Attach, Flow) antes del primer mensaje; se ven con `PN_TRACE_FRM=1` (Figura 3).
- **Sesiones y crédito:** el hub concede crédito al link y el cliente espera `on_sendable`; en MQTT no existe ese permiso explícito.
- **Tamaño percibido:** el payload JSON es el mismo; crece el encabezado binario AMQP (message-id, content-type, encoding, annotations).
- **Debug:** más difícil que paho-mqtt — no hay broker público para probar ni herramientas tipo MQTT Explorer; la traza de tramas es la herramienta principal.
- **Errores:** un mensaje mal formado vuelve `rejected` con causa, sin perder la conexión.

Qué problema resuelve AMQP mejor que MQTT: control de flujo y errores por mensaje en conexiones de larga duración con mucho volumen, y multiplexar muchos dispositivos de un gateway en una sola conexión TLS.

Limitación: Proton no implementa AMQP sobre WebSockets. Si el 5671 estuviera bloqueado, AMQP_WS (443) requiere el SDK de C#/Java/Node.

Evidencias:

```
[*] Cliente AMQP 1.0 (Qpid Proton) -> Azure IoT Central
[*] Host      : iotc-20ba21f7-4249-41e6-bb9c-55e4a8891e49.azure-devices.net:5671 (amqps / TLS)
[*] SASL      : PLAIN user=Vacuna-VM-Python@sas.iotc-20ba21f7-4249-41e6-bb9c-55e4a8891e49
[*] Link target : /devices/Vacuna-VM-Python/messages/events
[*] Mensajes   : 20 cada 5.0s
=====
[+] Conexión AMQP abierta (TCP+TLS+SASL+Open) en 680.0 ms
[+] Link sender adjuntado: /devices/Vacuna-VM-Python/messages/events
[+] Crédito concedido por el hub: 50 mensajes
[Msg #01] {'temperatura': 4.14, 'humedad': 51.94, 'nivel_bateria': 96.95} | Payload: 58 B | Disposition: ACCEPTED | Latencia: 208.36 ms
[Msg #02] {'temperatura': 3.99, 'humedad': 52.01, 'nivel_bateria': 96.9} | Payload: 57 B | Disposition: ACCEPTED | Latencia: 213.87 ms
[Msg #03] {'temperatura': 3.99, 'humedad': 52.17, 'nivel_bateria': 96.85} | Payload: 58 B | Disposition: ACCEPTED | Latencia: 206.84 ms
[Msg #04] {'temperatura': 4.08, 'humedad': 52.27, 'nivel_bateria': 96.8} | Payload: 57 B | Disposition: ACCEPTED | Latencia: 206.21 ms
[Msg #05] {'temperatura': 3.96, 'humedad': 52.02, 'nivel_bateria': 96.75} | Payload: 58 B | Disposition: ACCEPTED | Latencia: 206.22 ms
```

Figura 1. `device_amqp.py`: conexión abierta, crédito concedido por el hub y cada mensaje con `Disposition ACCEPTED` (~206 ms).

Devices > ContenedorVacunas > Vacuna-VM-Python

Vacuna-VM-Python
 Connected | Last data received: 10/6/2026, 7:36:24 AM | Status: Provisioned | Organization: Monitoreo-Vacunas-UNAB

Panel de Control del Operador Command Raw data Mapped aliases Files

Timestamp ↓	Message type	Event creation time	Humedad	Nivel de Batería	Temperatura
> 10/6/2026, 7:36:45 AM	Telemetry		51.09	96.35	4.43
> 10/6/2026, 7:36:40 AM	Telemetry		51.26	96.4	4.4
> 10/6/2026, 7:36:35 AM	Telemetry		51.53	96.45	4.31
> 10/6/2026, 7:36:30 AM	Telemetry		51.41	96.5	4.26
> 10/6/2026, 7:36:24 AM	Telemetry		51.26	96.55	4.29
> 10/6/2026, 7:36:19 AM	Telemetry		51.55	96.6	4.32
> 10/6/2026, 7:36:14 AM	Telemetry		51.84	96.65	4.21
> 10/6/2026, 7:36:09 AM	Telemetry		51.89	96.7	4.1
> 10/6/2026, 7:36:04 AM	Telemetry		52.02	96.75	3.96
> 10/6/2026, 7:35:58 AM	Telemetry		52.27	96.8	4.08
> 10/6/2026, 7:35:53 AM	Telemetry		52.17	96.85	3.99
> 10/6/2026, 7:35:48 AM	Telemetry		52.01	96.9	3.99
> 10/6/2026, 7:35:43 AM	Telemetry		51.94	96.95	4.14

Figura 2. Raw data de Vacuna-VM-Python (7:35–7:36 a. m.): las tres variables llegan cada 5 s por AMQP; estado Connected.

Figura 3 (evidencias/03_amqp_frames.txt , traza PN_TRACE_FRM=1):

```
AMQP:FRAME:0 <- @flow [incoming-window=0x1388, handle=0x0, delivery-
count=0x0, link-credit=0x32, drain=false]
AMQP:FRAME:0 <- @disposition [role=true, first=0x0, settled=true,
state=@accepted]
AMQP:FRAME:0 <- @disposition [role=true, first=0x1, settled=true,
state=@accepted]
```

La trama `@flow` es el hub concediendo 50 créditos; cada `@disposition ... @accepted` es la confirmación de un mensaje (equivalente al PUBACK de MQTT).

3. Etapa 2 — Tercer protocolo: HTTPS (REST)

Justificación de negocio: los registradores de los contenedores de vacunas pasan el viaje sin cobertura y descargan lo acumulado al llegar a una bodega o IPS, cuya red corporativa normalmente solo deja salir HTTPS por el 443 (a veces con proxy). Un POST sin sesión persistente encaja con ese patrón conectar-descargar-desconectar, y además llega a IoT Central de forma nativa.

`python-vm/device_https.py` hace, sin SDK, `POST` `https://{hub}/devices/{id}/messages/events?api-version=2021-04-12` con el SAS token en la cabecera `Authorization` y las propiedades `iothub-contenttype` / `iothub-contentencoding`. Respuesta esperada: `204 No Content`. La opción `--no-keepalive` abre un TLS nuevo por mensaje para medir el caso real de un dispositivo que se despierta, envía y se duerme.

Medición	Valor medido
Puerto	443 (TLS)
Payload JSON	57-58 B
Cabeceras HTTP de la petición	~569 B (~ 10 veces el payload; la mayor parte es el SAS token)
Latencia con keep-alive (20 msgs)	mediana 207,7 ms · p95 213,8 ms · primer mensaje 621 ms (incluye TLS)
Latencia sin keep-alive (10 msgs)	mediana 622,2 ms · p95 673,8 ms (TLS nuevo en cada POST)
Mensajes con HTTP 204	40 / 40 (en 3 corridas), 0 errores
¿Visible en Raw data?	Sí, las tres variables (Figura 5)

Ventajas: cero dependencias raras (`requests`), depurable con curl/Postman, atraviesa firewalls y proxies, sin estado en el dispositivo.

Desventajas: las cabeceras pesan ~10 veces el payload (569 B contra 58 B); sin canal persistente no hay comandos ni C2D en tiempo real (el dispositivo tendría que hacer polling); IoT Central muestra el dispositivo como desconectado aunque los datos llegan; cada POST sin keep-alive repite el handshake TLS y triplica la latencia (622 ms contra 208 ms).

Complejidad real: la más baja de los tres: unas 60 líneas con `requests`, reutilizando el generador de SAS token del Lab 3. Lo único no obvio son las cabeceras `iothub-contenttype` e `iothub-contentencoding`, necesarias para que Central mapee el JSON a las variables de la plantilla.

Evidencias:

```
=====
[*] Cliente HTTPS REST -> Azure IoT Central
[*] Endpoint      : POST https://iotc-20ba21f7-4249-41e6-bb9c-55e4a8891e49.azure-devices.net/devices/Vacuna-VM-Python/messages/events
[*] Puerto        : 443 (TLS)
[*] Keep-alive    : sí (requests.Session)
[*] Mensajes      : 20 cada 5.0s
=====
[Msg #01] {'temperatura': 4.25, 'humedad': 51.99, 'nivel_bateria': 96.95} | Payload: 58 B | Headers HTTP: ~569 B | HTTP 204 | Latencia: 621
32 ms
[Msg #02] {'temperatura': 4.29, 'humedad': 51.97, 'nivel_bateria': 96.9} | Payload: 57 B | Headers HTTP: ~569 B | HTTP 204 | Latencia: 207.
27 ms
[Msg #03] {'temperatura': 4.36, 'humedad': 52.25, 'nivel_bateria': 96.85} | Payload: 58 B | Headers HTTP: ~569 B | HTTP 204 | Latencia: 207
99 ms
[Msg #04] {'temperatura': 4.51, 'humedad': 52.43, 'nivel_bateria': 96.8} | Payload: 57 B | Headers HTTP: ~569 B | HTTP 204 | Latencia: 200.
21 ms
|
```

Figura 4. `device_https.py`: cada POST devuelve HTTP 204; el primero tarda 621 ms por el handshake TLS y los siguientes ~207 ms al reutilizar la conexión.

Timestamp ↓	Message type	Event creation time	Humedad	Nivel de Bateria	Tempera
> 10/6/2026, 7:38:05 AM	Telemetry		52.49	96.6	4.33
> 10/6/2026, 7:37:59 AM	Telemetry		52.55	96.65	4.32
> 10/6/2026, 7:37:54 AM	Telemetry		52.31	96.7	4.47
> 10/6/2026, 7:37:49 AM	Telemetry		52.33	96.75	4.39
> 10/6/2026, 7:37:44 AM	Telemetry		52.43	96.8	4.51
> 10/6/2026, 7:37:38 AM	Telemetry		52.25	96.85	4.36

Figura 5. Raw data (7:37–7:38 a. m.): los mensajes enviados por HTTPS llegan con las mismas tres variables.

4. Etapa 3 — Comparación MQTT · AMQP · HTTPS

Con la conexión ya abierta, AMQP y HTTPS tardan lo mismo (~207 ms por mensaje, la ida y vuelta Codespaces–Azure); lo que los separa es el coste de conectar, las cabeceras y lo que permiten hacer. MQTT sigue siendo el mejor protocolo de dispositivo; AMQP gana entre servicios y HTTPS solo sirve para envíos esporádicos detrás de un firewall. Las cifras de MQTT vienen del Lab 3 y las de AMQP y HTTPS de las corridas del 6 oct 2026 (`evidencias/RESULTADOS.txt`).

Criterio	MQTT (Lab 3, paho)	AMQP 1.0 (Proton)	HTTPS REST
Puerto / transporte	8883 TLS (443 con WS)	5671 TLS (443 con WS, no disponible en Python)	443 TLS

Criterio	MQTT (Lab 3, paho)	AMQP 1.0 (Proton)	HTTPS REST
Overhead por mensaje	Mínimo: cabecera fija de 2 B + tópico; JSON 61-64 B	Medio: cabecera binaria con message-id, content-type y anotaciones; JSON 58 B	Alto: ~569 B de cabeceras HTTP por 58 B de JSON
Coste de conexión	CONNECT + CONNACK, una vez	680 ms una vez (TLS + SASL + Open + Begin + Attach + Flow)	~620 ms por mensaje sin keep-alive; una vez con keep-alive
Latencia por mensaje	120-220 ms	mediana 206,4 ms · p95 213,9 ms	207,7 ms con keep-alive · 622,2 ms sin keep-alive
Fiabilidad	QoS 0/1, sin QoS 2 en Azure; un error cierra la conexión	46/46 aceptados; disposición por mensaje (accepted / rejected / released) + 50 créditos	40/40 con HTTP 204; reintentar queda a cargo del cliente
Soporte MCU / Wokwi	Excelente (PubSubClient ya funcionó en el ESP32)	Malo: no hay cliente AMQP ligero estable para ESP32	Posible (HTTPClient + TLS), pero pesado en RAM
Debug	Fácil (MQTT Explorer, logs de paho)	Difícil (traza binaria <code>PN_TRACE_FRM</code>)	Muy fácil (curl, Postman)
Paso de firewalls	Solo con MQTT-WS	Solo con AMQP-WS	Sí, es tráfico web normal
Comandos / twin desde Central	Sí	Sí	No (solo polling de C2D)
Caso de uso ideal	Sensor conectado que reporta cada pocos segundos	Gateway con muchos dispositivos; servicios hacia Service Bus / Event Hubs	Logger que sube un lote al llegar a la bodega

5. Recomendaciones por capa

- **Dispositivo (Wokwi / ESP32 futuro): MQTT sobre TLS, QoS 1.** Ya funciona en el ESP32 del Lab 3, tiene el menor overhead para el plan de datos celular y permite comandos de Central. Fallback MQTT-WS en el 443 si la red del cliente bloquea el 8883.
- **Entre servicios Azure / plataforma propia (Labs 5-8): AMQP.** Es lo que hablan Service Bus y Event Hubs; el crédito y la disposición por mensaje dan backpressure y manejo de errores sin perder la conexión. Se usará vía los SDK de servicio (`azure-servicebus` , `azure-eventhub`), no a mano.
- **Los dos protocolos para el resto del curso: MQTT (borde) + AMQP (backend).** Cubren las dos capas de la cadena de frío sin repetirse. HTTPS queda como herramienta de diagnóstico y para cargas por lote desde bodegas, no como transporte principal: su overhead por mensaje y la falta de comandos lo descartan para telemetría continua.

Estructura del repositorio

```
labIotJuanRojas4/
├── README.md
├── evidencias/          # capturas + amqp_*.jsonl, https_*.jsonl,
03_amqp_frames.txt
├── python-vm/
│   ├── common.py       # credenciales por env, SAS token, sensor, logger
│   ├── device_amqp.py  # Etapa 1 (Qpid Proton)
│   ├── device_https.py # Etapa 2 (requests)
│   ├── provision.py    # hostname vía DPS por HTTPS (opcional)
│   ├── resumen.py      # mediana / p95 para la tabla
│   └── requirements.txt
```

Fuentes

- [Communicate with your IoT hub by using the AMQP Protocol — Microsoft Learn](#)
- [Communicate with an IoT hub using the MQTT protocol \("AMQP isn't supported in the Python SDK"\) — Microsoft Learn](#)
- OASIS AMQP 1.0 (sesiones, links, control de flujo por créditos)